Design and Analysis of Algorithms

- ***Course Code***: BCSE204L
- ***Course Type***: Theory (ETH)
- ***Slot***: A1+TA1 & & A2+TA2
- ***Class ID***: VL2023240500901
  VL2023240500902

A1+TA1

Day	Start	End
Monday	08:00	08:50
Wednesday	09:00	09:50
Friday	10:00	10:50

A2+TA2

Day	Start	End
Monday	14:00	14:50
Wednesday	15:00	15:50
Friday	16:00	16:50

Syllabus- Module 5

Module:5	Geometric Algorithms	4 hours

- Line Segments: Properties, Intersection, **sweeping lines** - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.

Geometric Algorithms:

Determining whether any pair of segments intersect

- In geometric algorithms, determining whether any pair of segments intersect involves designing an algorithm to ascertain if any two line segments in a given set intersect.
- This algorithm typically utilizes a technique known as "sweeping," which is common in many computational geometry algorithms.
- The algorithm operates with a time complexity of O(nlogn),
  - where n represents the number of segments provided.

Note:

- Its purpose is to determine only whether or not any intersections exist, without printing all the intersections.
- It is possible to develop an algorithm to print all the intersections in O(n^2) time in the worst case scenario.

Geometric Algorithms: Determining whether any pair of segments intersect

Determining whether any pair of segments intersect using a **naive approach** involves checking every possible pair of segments for intersection, which leads to a time complexity of O(n^2).
Because

- It involves checking every pair of segments for intersection.
- For each pair of segments, you can use geometric formulas to determine if they intersect.
- Since there are n choose 2 = n(n-1)/2 pairs of segments in a set of n segments, the time complexity of this approach is O(n^2) which can be slow for large datasets.

However, this approach becomes inefficient for large datasets due to its quadratic time complexity.

In contrast, utilizing the "**sweeping**" technique enables the development of an algorithm with a significantly improved time complexity of **O(n log n)**.

- By moving a sweep line across the plane and efficiently handling events as it encounters endpoints of segments, the algorithm can determine segment intersections more efficiently.
- This makes it suitable for handling larger datasets within a reasonable amount of time.

Geometric Algorithms: What is Sweeping?

- "**Sweeping**" is a technique commonly used in computational geometry to solve various geometric problems efficiently.
- The basic idea of sweeping involves moving a line or plane (the "sweep line" or "sweep plane") across the geometric space while keeping track of events or changes that occur as the line moves.
- This approach is particularly useful for problems involving arrangements of geometric objects, such as points, line segments, or polygons.

What is Sweeping Line?

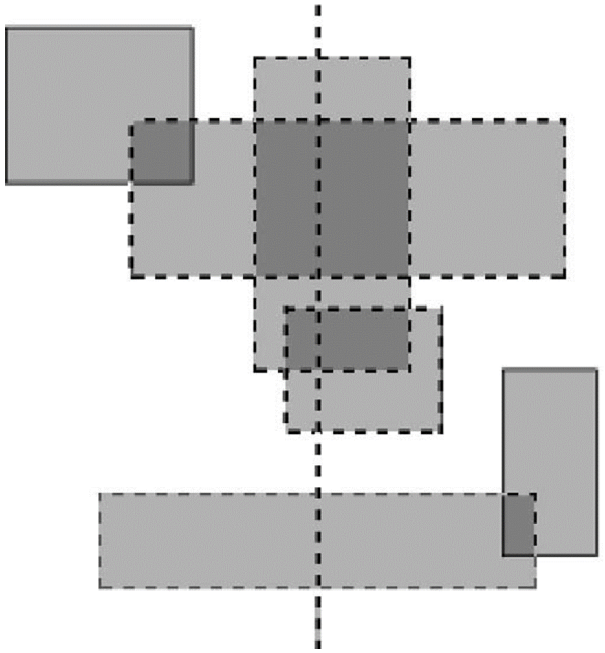
The sweep line algorithm involves moving a vertical line (the sweep line) across the plane horizontally from left to right, processing events as it encounters them. These events can include the start or end of line segments, intersections, or other geometric features.

- The use of sweeping lines enables efficient processing of geometric data by exploiting the geometric properties induced by the movement of the sweep line.
- It reduces the complexity of geometric algorithms by avoiding the need to consider all possible pairs of geometric objects, instead focusing on events that occur along the sweep line.

Geometric Algorithms: What is Sweeping?

What is Sweeping Line?

The sweep line algorithm involves moving a vertical line (the sweep line) across the plane horizontally from left to right, processing events as it encounters them. These events can include the start or end of line segments, intersections, or other geometric features.



- **Vertical Line:** Imagine a straight line standing upright, perpendicular to the ground (like a door). In the sweep line algorithm, this imaginary line moves sideways, not up and down.
- **Horizontal Movement:** This vertical line conceptually sweeps across the plane from left to right.

Geometric Algorithms: Determining whether any pair of segments intersect

Sweeping Technique: Sweeping involves the traversal of an imaginary vertical sweep line across a given set of geometric objects, typically from left to right. This technique facilitates the ordering of geometric objects and allows the exploitation of relationships among them.

Methodology:

- **Ordering Geometric Objects**: One of the primary purposes of sweeping is to order geometric objects along the sweep line. This ordering is often achieved by employing dynamic data structures such as **red-black trees**.
- **Taking Advantage of Relationships**: By organizing geometric objects along the sweep line, the algorithm can efficiently detect and analyze relationships among them.

Note: Exploiting Relationships: Leveraging the ordered data to identify and analyze relationships between the objects, such as intersections or overlapping areas.

Geometric Algorithms: Determining whether any pair of segments intersect

Line-Segment-Intersection Algorithm:

- **Simplifying Assumptions**: The algorithm typically operates under simplifying assumptions to streamline the intersection detection process. Two common assumptions are:
  1. No input segments are vertical.
     - Vertical line segments require special handling to determine intersections with other segments. By excluding them, the algorithm can focus on horizontal and non-vertical segments, making the calculations and comparisons easier.
  2. No three or more input segments intersect at a single point.
     - This assumption avoids dealing with the complex scenario. By assuming no such intersections, the algorithm focuses on standard pairwise intersections between two segments.

Modification for Assumptions: While the algorithm can be adapted to handle cases that violate these assumptions, doing so may significantly increase its complexity, particularly in terms of correctness verification.

Geometric Algorithms: Determining whether any pair of segments intersect

Ordering Segments:

- **Handling Vertical Segments**: Since vertical segments are excluded from the input, any segment intersecting a vertical sweep line does so at a single point. This property enables ordering segments based on the y-coordinate of their intersection point with the sweep line.
- **Comparison of Segments**: Segments are deemed comparable at a specific x-coordinate if the vertical sweep line intersects both segments at that point. The relationship between two segments at a given x-coordinate is determined by comparing their intersection points with the sweep line.

Note:

- **Y-coordinate comparison:** The segment with the lower y-coordinate (intersecting lower on the sweep line) is considered "above" the other segment.
- **Slope comparison (optional)**: In some variations, the algorithm might also consider the slopes of the segments to determine their relative positions (e.g., one segment going upwards and the other downwards).

Geometric Algorithms: Determining whether any pair of segments intersect

How the sweep line moves and how events are managed in sweeping algorithms.

Managing Sweep Line Data:

- **Sweep Line Status**: This data structure maintains the current state of the sweep line and provides information about the relationships among the objects intersected by the sweep line. It's typically updated only at event points.
- **Event Point Schedule**: The event point schedule is a sequence of points ordered from left to right along the sweep line. These points correspond to events such as the endpoints of line segments. When the sweep line reaches an event point, it halts, processes the event, and then resumes sweeping.

Sorting Segment Endpoints:

- Segment endpoints are sorted by increasing x-coordinate. In case of tie (i.e., co-vertical endpoints), additional rules are applied to break the tie:
  - Co-vertical left endpoints are placed before co-vertical right endpoints.
  - Within co-vertical endpoints, those with lower y-coordinate are placed first.
- This sorting process ensures consistent handling of segment endpoints during the sweep.

Geometric Algorithms: Determining whether any pair of segments intersect

How the sweep line moves and how events are managed in sweeping algorithms.

Moving the Sweep Line:

- **Insertion and Deletion of Segments**: When encountering the left endpoint of a segment, it is inserted into the sweep line status. Conversely, when encountering the right endpoint of a segment, it is deleted from the sweep line status.
- **Intersection Checking**: When inserting or deleting segments, it's crucial to check for intersections between the newly inserted/deleted segment and its neighboring segments. This involves checking whether two consecutive segments intersect for the first time, known as the "ABOVE" and "BELOW" operations.

By managing the sweep line status and event points effectively, the algorithm can accurately track the state of geometric objects and efficiently detect intersections as the sweep line progresses.

Geometric Algorithms: Determining whether any pair of segments intersect

```
Algorithm ANY-SEGMENTS-INTERSECT(S)
Input: Set S of n line segments;  Output: TRUE if any pair of segments in S intersect, FALSE otherwise

T = Empty Red-Black Tree  // Initialize an empty red-black tree to maintain the sweep line status

SortEndpoints(S)  // O(n log n) //Sort the endpoints of line segments in S from left to right

// Iterate through each endpoint in the sorted list
for each endpoint p in the sorted list of endpoints  // O(n)
    if p is the left endpoint of segment s
        InsertSegment(T, s)  // O(log n)
        if IntersectionWithAbove(T, s) or IntersectionWithBelow(T, s)
            return TRUE
    else if p is the right endpoint of segment s
        if IntersectionBetweenAboveAndBelow(T, s)
            return TRUE
        DeleteSegment(T, s)  // O(log n)

return FALSE // If no intersections found during the sweep, return FALSE
```

Geometric Algorithms: Determining whether any pair of segments intersect

```
Function SortEndpoints(S)
    // Sort endpoints based on x-coordinate (break ties by y-coordinate)
    // Implementation not shown, typically using merge sort or heap sort
```

```
Function InsertSegment(T, s)
    // Insert segment s into the red-black tree T
    // Implementation not shown, typically using tree insertion operations
```

```
Function DeleteSegment(T, s)
    // Delete segment s from the red-black tree T
    // Implementation not shown, typically using tree deletion operations
```

```
Function IntersectionWithAbove(T, s)
    // Check if segment s intersects with its above neighbor in tree T
    // Implementation not shown, typically involves comparing s with the segment immediately above it in T
```

```
Function IntersectionWithBelow(T, s)
    // Check if segment s intersects with its below neighbor in tree T
    // Implementation not shown, typically involves comparing s with the segment immediately below it in T
```

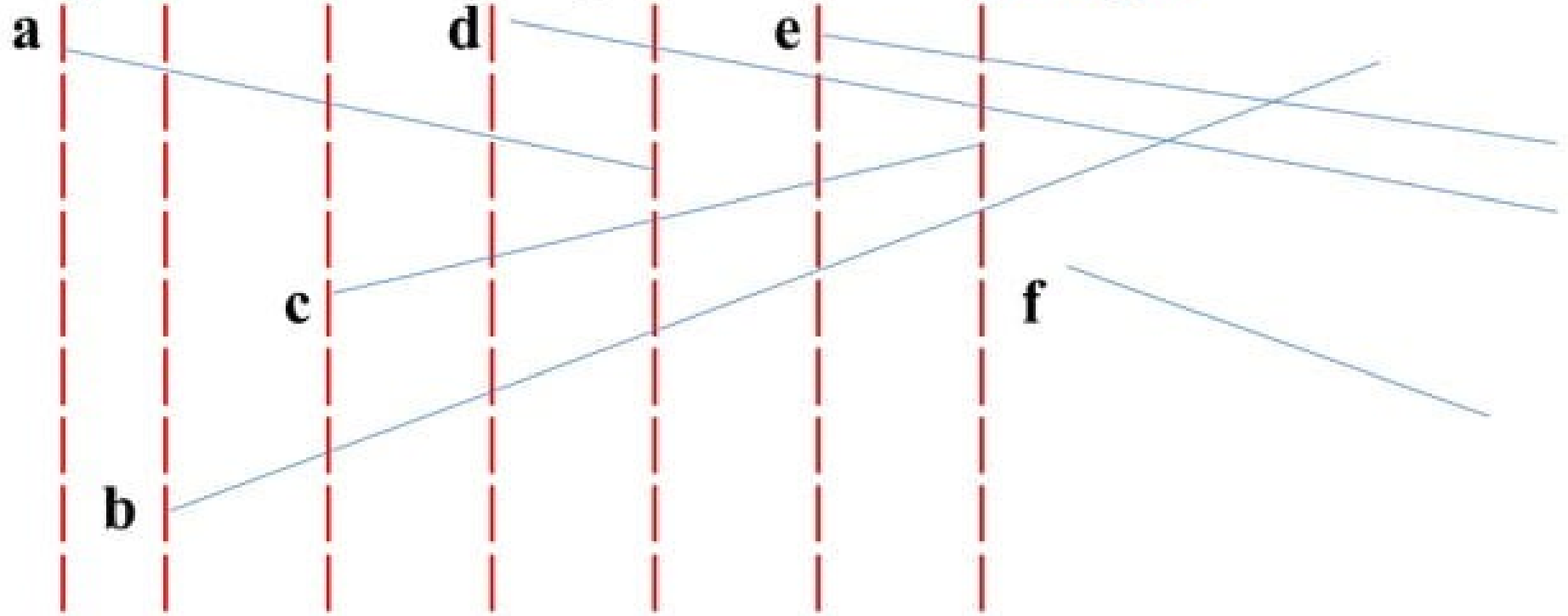
Geometric Algorithms: Determining whether any pair of segments intersect

1. **Initialization**:
    1. Initialize an empty red-black tree (or any suitable data structure) to maintain the sweep line status, denoted as T.
2. **Sorting Endpoints**:
    1. Sort the endpoints of the line segments in S from left to right. Ensure tie-breaking rules: left endpoints before right endpoints, and for co-vertical endpoints, lower y-coordinate first. This sorting step takes O(n log n) time using merge or heapsort.
3. **Sweeping Process**:
    1. Iterate through each endpoint in the sorted list.
    2. If the endpoint is a left endpoint of a segment:
        1. Insert the segment into the sweep line status T using the INSERT operation, which takes O(log n) time.
        2. Check if the segment intersects with its above or below neighbor segments in T. If so, return TRUE to indicate an intersection.
    3. If the endpoint is a right endpoint of a segment:
        1. Delete the segment from T using the DELETE operation, which also takes O(log n) time.
        2. Check if both the above and below neighbors of the segment intersect with each other. If they do, return TRUE.
4. **Return False**:
    1. If no intersections are found during the sweep, return FALSE to indicate that no pair of segments intersect.
5. **Time Complexity**:
    1. The time complexity of the algorithm is dominated by the sorting step and the operations on the red-black tree (insertion and deletion), resulting in O(n log n) overall.

Unit IV : Geometric Algorithms

Determining whether any pair of segments intersect :

Segment intersection pseudo code : Example



a	a	a	d	d	e	Algorithm ends with TRUE
	b	c	a	c	d	since segments d and b
		b	c	b	c	intersect when segment c
			b		b	deleted d and b become
						consecutive first time.

Any Question?

Thank You!

FOR YOUR ATTENTION